
mqtt-recorder
mosquitto broker
gateway (GatewayApplication)
EnergyThermal-SensorPublisher
Backend MqttConfig
MqttMessageService
StationRegistery
LogService
SignalFactory
LiveSync
Signal
Incoming readings
publish "PLC/Island 1/<Type>/Station/.../<signal>"
deliver message (subscribed "PLC/Island 1/#")
updateExecuteingState(topic, payload)
(update EXECUTING_STATE map if isExecuting)
newTopic = topic.replace("PLC", "DigitalTwin")
publish(newTopic, same payload
loop (2 seconds)
read EXECUTING_STATE
compute temp & power (heating/cooling)
publish "DigtalTwin/.../measurements/Internal/currentPowerInWatt"
publish "DigtalTwin/.../measurements/Internal/currentTempCelsius"
deliver "DigitalTwin/Island 1/#"
handleMessage(topic, payload)
Process(topic, Payload)
parse stationName from Topic
getStationByName(stationName)
station
alt
topic == "events"
handleStationEvents()
direction == "received" && eventType == "command""
logCommandEvent(stationName, commandId, commandName, parameters, timeStamp)
direction == "emitted"
eventType == "command_feedback"
handleCommandFeedback()
logCommandFeedback(stationName, commandId, machineCommandStatus, timeStamp)
eventType == "machine_feedback"
handleMachineFeedback()
logStatus(stationName, stationsState, message, timeStamp)
regular signal
handleSignal(station, parts, payload, topic)
resolveSignal(station, parts, topic)
signal not known
build(parts, topic, station)
create Signal, attach to station
queue(key, ()->{setCurrentValue(value); setLastUpdated(timeStamp);})
logMeasurement(stationName, signalName, signalCategory, signalKind, topic, value, timeStamp)
updateStationFields(station, SignalName, value)
signalName == "currentTempCelsius"
queue(key, ()->{setCurrentTempCelsius(value); evalTemps(station, value);})
signalName == "currenPowerInWatt"
queue(key, ()->{setCurrentPowerInWatt(value);})
signalName == "isExecuting"
queue(key, ()->{setIsExecuting(value);})
station-type specific sensor/actuator
queue(key, ()->{set<Field>(value);})